GNOME Shell Extensions: A Beginner's Guide (TypeScript)

Oct 11, 2026 · @Greenal

How to use this guide

This guide teaches GNOME Shell extension development from zero, in TypeScript, and ends by building an i3-style window list at the bottom of the screen. Every chapter builds on the one before it, so read Chapters 1 to 7 in order; after that, jump to whatever you need.

Who it's for: you know basic JavaScript or TypeScript, you have followed the gjs.guide TypeScript setup, and the official docs feel like a pile of disconnected reference pages.

Which GNOME: written against GNOME Shell 49 to 51 (51 is current as of October 2026). Where an API changed recently, the chapter says so. Run `gnome-shell --version` to check yours.

Conventions used here:

- `global`, `Main`, `St` and friends are explained in Chapter 1; if a name confuses you, look there first.
- Code is TypeScript. Remove the type annotations and it is valid GJS JavaScript.
- "Actor" means anything drawn on screen (Chapter 8). "Signal" means an event you can listen to (Chapter 6).
- Lines marked `// ❌` show a common mistake; `// ✅` shows the fix.

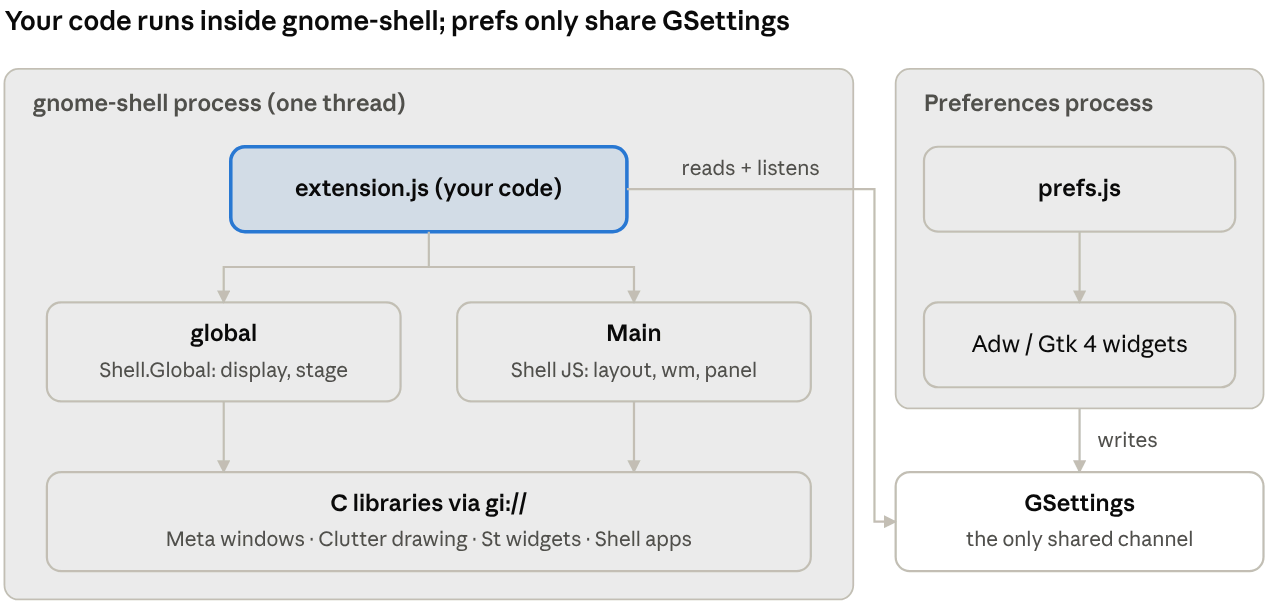
Reading order:

If you want to…	Read
Understand what you are even working with	1, 5
Get a working dev loop	2, 3
Stop leaking memory and crashing the shell	4, 6
Query windows, focus, workspaces	7
Draw a panel and buttons	8, 9

If you want to…	Read
Add settings and shortcuts	10, 11
Build the window list end to end	13

Chapter 1 — The big picture

An extension is a JavaScript module that GNOME Shell loads into its own process and lets you change the live desktop. There is no sandbox and no separate app: your code runs in the same process that draws the top bar, the overview and every window frame. That is why extensions are so powerful, and why a bug can freeze your whole session.



how an extension fits into GNOME Shell · 2 processes

Your extension reaches the desktop through `global` and `Main`, which sit on top of the C libraries. The preferences window is a separate app and can only talk to your extension through GSettings.

1.1 What GNOME Shell actually is

GNOME Shell is a program written in two halves:

- **C libraries** do the heavy lifting: Mutter manages windows and composites them, Clutter draws everything, St provides widgets.

- **JavaScript** (in the `gnome-shell` source under `js/ui/`) builds the actual UI you see: the panel, the overview, the app grid, popup menus, notifications.

The JavaScript runs on **GJS**, GNOME's JavaScript engine (Firefox's SpiderMonkey plus glue). Your extension is just more JavaScript running in the same GJS engine, next to Shell's own code. You can call into the C libraries, and you can import and even modify Shell's JavaScript modules.

1.2 How JavaScript talks to C: GObject Introspection

The C libraries are not written for JavaScript. A system called **GObject Introspection (GI)** reads metadata about each C library and generates JavaScript bindings at runtime. That is what `gi://` imports are:

```
import Meta from "gi://Meta";   // the Mutter C library, as JS
import St from "gi://St";       // the St C library, as JS
```

Two consequences that explain a lot of confusion:

1. The API reference at gjs-docs.gnome.org is **generated from C documentation**. It reads like C because it is C, translated. Chapter 5 teaches you how to read it.
2. Everything from a `gi://` import is a **GObject**: it has properties, methods and signals in a uniform way. Learn the GObject pattern once (Chapter 6) and it applies to every library.

1.3 The libraries you will use

Import	What it is	Typical things you use it for
`gi://Meta`	Mutter, the window manager and compositor	`Meta.Window`, `Meta.Display`, workspaces, monitors, focus, keybinding flags
`gi://Mtk`	Mutter toolkit, small helpers	`Mtk.Rectangle` (replaced `Meta.Rectangle`, removed in GNOME 49)
`gi://Clutter`	The scene graph: everything drawn is a `Clutter.Actor`	layout, alignment, animation modes, input events and gestures

Import	What it is	Typical things you use it for
`gi://St`	Shell Toolkit: CSS-styled widgets built on Clutter	`St.Button`, `St.Label`, `St.BoxLayout`, `St.Icon`, `St.Bin`
`gi://Shell`	C helpers specific to GNOME Shell	`Shell.App`, `Shell.AppSystem`, `Shell.WindowTracker`, `Shell.ActionMode`
`gi://GObject`	The object system everything is built on	registering classes, properties, signals, binding flags
`gi://GLib`	Low-level utilities	timers, the main loop, `GLib.Variant`, time constants
`gi://Gio`	I/O and system services	`Gio.Settings` (GSettings), files, D-Bus, subprocesses
`gi://Adw`, `gi://Gtk`	GTK 4 and libadwaita	**only** in `prefs.ts` (the preferences window)
`resource:///org/gnome/shell/ui/*.js`	GNOME Shell's own JavaScript	`Main`, `PopupMenu`, `PanelMenu`, `DND`, `Dash` and more
`resource:///org/gnome/shell/extensions/extension.js`	The extension base class	`Extension`, `gettext`

One hard rule: **never import Gtk, Gdk or Adw in `extension.ts`**, and never import Clutter, Meta, St or Shell in `prefs.ts`. They run in different processes (Chapter 2) and mixing them crashes.

1.4 `global` and `Main`: your two entry points

You almost never construct the important objects yourself. GNOME Shell already created them; you reach them through two doors.

`global` is a `Shell.Global` object that the Shell injects into every JavaScript file. It is not imported. It hands you the core C-side singletons:

Expression	Gives you
`global.display`	the `Meta.Display`: focus, windows, monitors, the tab list
`global.workspace_manager`	the `Meta.WorkspaceManager`: list and switch workspaces
`global.window_manager`	the `Shell.WM`: signals like `switch-workspace`
`global.stage`	the `Clutter.Stage`, root of everything drawn
`global.get_window_actors()`	one `Meta.WindowActor` per window; `.metaWindow` gives the window
`global.get_current_time()`	the event timestamp that `activate()` and `delete()` need
`global.compositor`, `global.backend`	lower-level Mutter objects

`Main` (`import * as Main from "resource:///org/gnome/shell/ui/main.js"`) holds GNOME Shell's JavaScript singletons:

Expression	Gives you
`Main.layoutManager`	monitors, adding your own on-screen chrome
`Main.panel`	the top bar
`Main.overview`	the Activities overview, with `showing` and `hidden` signals
`Main.wm`	the JS window manager: `addKeybinding`, `removeKeybinding`
`Main.uiGroup`	the actor that holds all Shell UI; popup menus go here
`Main.notify()`	send a quick notification

Rule of thumb: **C-level desktop state comes from `global`; Shell UI comes from `Main`.**

1.5 Why `Meta.Display.get_focus_window()` fails

`Meta.Display` is a **class**. `get_focus_window()` is an **instance method**: it needs a real display object to ask. There is exactly one real display, and Shell created it for you:

```
Meta.Display.get_focus_window();       // ❌ calling on the class
global.display.get_focus_window();     // ✅ calling on the instance
global.display.focus_window;           // ✅ same thing, as a property
```

The docs always show the class. Your job is to find the instance, and `global` or `Main` is almost always where it lives.

Chapter 2 — Project anatomy (TypeScript)

GNOME Shell only runs JavaScript, so a TypeScript extension has two folders: your **source** folder, where you write `.ts`, and the **installed** folder, where the compiled `.js` lives. The build step turns one into the other.

2.1 The source folder

```
my-extension/
├── metadata.json          # identity card, required
├── extension.ts           # runs inside gnome-shell
├── prefs.ts               # optional: preferences window (separate process)
├── stylesheet.css         # optional: CSS for your St widgets
├── schemas/
│   └── org.gnome.shell.extensions.my-extension.gschema.xml
├── ambient.d.ts           # makes gi:// and resource:// imports typed
├── tsconfig.json
├── package.json
└── Makefile
```

2.2 The installed folder

GNOME Shell looks for user extensions in `~/.local/share/gnome-shell/extensions/<uuid>/`. After `make install` it contains:

```
~/.local/share/gnome-shell/extensions/my-extension@example.com/
├── metadata.json
├── extension.js          # compiled from extension.ts
├── prefs.js
├── stylesheet.css        # loaded automatically if present
└── schemas/
    ├── org.gnome.shell.extensions.my-extension.gschema.xml
    └── gschemas.compiled  # made by glib-compile-schemas
```

The file names `extension.js`, `prefs.js` and `stylesheet.css` are fixed. Shell looks for exactly those.

2.3 `metadata.json`

```
{
  "uuid": "app-tab@yourname.github.io",
  "name": "App Tab",
  "description": "An i3-style window list at the bottom of the screen",
  "shell-version": ["49", "50", "51"],
  "url": "https://github.com/yourname/app-tab",
  "settings-schema": "org.gnome.shell.extensions.app-tab"
}
```

Key	Meaning
`uuid`	Globally unique id, form `name@domain`. Must match the install folder name exactly.
`shell-version`	GNOME versions you have tested. Shell refuses to load the extension on any other major version.
`settings-schema`	Lets `this.getSettings()` find your schema with no arguments.
`url`	Your repository; required for publishing.
`version`	Don't set it; extensions.gnome.org assigns it.

The most common "my extension doesn't show up" bug is a `uuid` that doesn't match the folder name, or a `shell-version` that doesn't include your GNOME.

2.4 `extension.ts`: the minimum

```
import { Extension } from
"resource:///org/gnome/shell/extensions/extension.js";

export default class AppTab extends Extension {
  enable(): void {
    // create things, connect signals
  }

  disable(): void {
    // destroy everything enable() made, disconnect everything
  }
}
```

Three requirements: a **default export**, a class that **extends `Extension`**, and both **`enable()`** and **`disable()`**. Don't put setup in the constructor; Shell may construct the class once and call `enable()` / `disable()` many times (Chapter 4).

The `Extension` base class gives you useful members:

Member	What it is
`this.getSettings()`	your `Gio.Settings` (Chapter 10)
`this.metadata`	the parsed `metadata.json`
`this.uuid`	your uuid
`this.path`, `this.dir`	the install folder, as a string and a `Gio.File`
`this.openPreferences()`	opens your prefs window

2.5 Two processes

`extension.js` runs **inside gnome-shell**. `prefs.js` runs in a **separate GTK app** (`gnome-extensions-app` or the Extensions settings) when the user opens preferences. They share nothing except GSettings. That's why the settings system is how the preferences window talks to your extension: prefs writes a key, your extension listens for the change.

2.6 TypeScript specifics

- `ambient.d.ts` imports the `@girs/*` type packages so that `gi://Meta` and `resource:///...` imports have types, and so that `global` is declared.
- The `@girs` packages are versioned. Keep `@girs/gnome-shell` close to your installed GNOME version, or the types will describe APIs you don't have (or lack ones you do).
- Import paths stay `gi://…` and `resource:///…` in your `.ts` files. TypeScript must not rewrite them; the guide's `tsconfig.json` (`module: NodeNext`) leaves them alone.
- Shell's own JS modules are typed by `@girs/gnome-shell`, but those types are hand-written and sometimes incomplete. When a type is wrong, a narrow cast is fine: `(Main.layoutManager as any).someField`. Prefer checking the source first (Chapter 5).
- Don't use `Boolean`, `String`, `Number` (wrapper types). Use `boolean`, `string`, `number`.

2.7 Build and install

```
make            # npm install if needed, then tsc → dist/
make install    # compile schemas, zip, gnome-extensions install --force
gnome-extensions enable app-tab@yourname.github.io
```

On Wayland (the only session since GNOME 50 removed X11), a running Shell **cannot reload** an extension's code. After installing new code you either log out and back in, or test in a devkit session (next chapter). Plan your workflow around this.

Chapter 3 — The development loop

The fastest safe loop is: edit, `make install`, start a devkit Shell in a window, read the logs. Your real desktop never restarts and a crash only kills the test window.

3.1 Test in a devkit session

Since GNOME 49 the old `--nested` mode is gone. Use the development kit instead:

```
make install
dbus-run-session gnome-shell --devkit --wayland
```

This opens a full GNOME Shell inside a window on your desktop. It reads the same extensions folder, so your freshly installed code is loaded. To try a new build: close the window, `make install`, run the command again. If the devkit viewer is missing, install your distro's `mutter-devkit` (or similarly named) package.

Inside that session, enable your extension once:

```
# in a terminal inside the devkit session, or before launching it
gnome-extensions enable app-tab@yourname.github.io
```

3.2 Read the logs

Everything your extension prints, and every exception it throws, goes to the system journal.

```
# real session
journalctl -f -o cat /usr/bin/gnome-shell
```

For a devkit session, its output appears in the terminal you launched it from. Write to the log with:

```
console.log("panel created", this._panel);    // normal
console.debug("focus changed");               // verbose; may be hidden by
default
console.warn("no primary monitor");
console.error(err);                           // includes the stack trace
```

When an error happens in `enable()`, Shell marks the extension as **errored** and disables it. `gnome-extensions info <uuid>` shows the state and the error.

3.3 Looking Glass: a live console

Press **Alt+F2**, type `lg`, press Enter. You get a JavaScript console running inside the Shell, with `global` available. It is the best way to explore an API before writing code:

```
global.display.focus_window.title
global.display.focus_window.get_wm_class()
global.get_window_actors().map(a => a.metaWindow.title)
global.workspace_manager.get_active_workspace_index()
```

Two more tabs are worth knowing: **Picker** (the eyedropper at the top left) lets you click any piece of Shell UI and inspect that actor; **Extensions** lists loaded extensions and their errors. Press **Escape** to close.

If `Main` is not defined in your version's Looking Glass, load it with `const Main = await import("resource:///org/gnome/shell/ui/main.js")`. `r(0)` recalls result number 0, so you can keep poking at an object you found earlier.

3.4 Common first errors

Symptom	Usual cause
Extension not in the list	`uuid` doesn't match folder name, or `shell-version` excludes your GNOME
Listed but state is ERROR	Exception in `enable()` or at import time; read the journal
`ImportError` / module not found	A path in a `resource:///` import is wrong, or TypeScript rewrote it
`x is not a function` on a `gi://` class	You called an instance method on the class (Chapter 1.5)
Settings: "schema not found"	Forgot `glib-compile-schemas`, or the schema id doesn't match `settings-schema`
Works once, breaks after lock screen	`disable()` doesn't fully undo `enable()` (Chapter 4)
Panel invisible	Actor has no size, no background, or was never added to the stage

Chapter 4 — The extension lifecycle

`disable()` must leave the Shell exactly as it was before `enable()`. This is the single most important rule in extension development, and the first thing reviewers check.

4.1 When enable and disable run

Your extension is not only enabled at login and disabled at logout. Shell also calls `disable()` when the **screen locks** and `enable()` again when it unlocks (the lock screen runs in a different session mode where most extensions are off). It is also toggled when the user flips the switch in the Extensions app, and when the extension is updated.

So in a normal day your `enable()` / `disable()` pair may run dozens of times. Anything you forget to clean up gets **duplicated** each cycle: two panels, three signal handlers, a growing pile of timers.

4.2 The cleanup checklist

For every line in `enable()`, there is a matching line in `disable()`:

In `enable()` you…	In `disable()` you…
create an actor	`actor.destroy()`
`Main.layoutManager.addChrome(actor)`	`Main.layoutManager.removeChrome(actor)` then destroy
`obj.connect(...)` (returns an id)	`obj.disconnect(id)`
`obj.connectObject(..., this)`	`obj.disconnectObject(this)`
`GLib.timeout_add(...)` (returns an id)	`GLib.source_remove(id)`
`Main.wm.addKeybinding(name, ...)`	`Main.wm.removeKeybinding(name)`
`this.getSettings()`	set your reference to `null`
change a Shell object (hide the panel, patch a method)	put it back exactly
store any object on `this`	set the field to `null` / `undefined`

That last row matters for garbage collection. A reference you keep after `disable()` keeps a destroyed actor alive, and touching a destroyed actor later logs "Object has already been disposed" errors.

4.3 A safe pattern

Put each piece of state in one field, and make `disable()` tolerant of partial setup (if `enable()` threw halfway, `disable()` must still work):

```
export default class AppTab extends Extension {
  private _settings: Gio.Settings | null = null;
  private _panel: St.BoxLayout | null = null;
  private _timeoutId = 0;

  enable(): void {
    this._settings = this.getSettings();
    this._panel = new St.BoxLayout();
    Main.layoutManager.addChrome(this._panel);
    global.display.connectObject("notify::focus-window", () => this._sync(),
this);
  }

  disable(): void {
    global.display.disconnectObject(this);
    if (this._timeoutId) {
      GLib.source_remove(this._timeoutId);
      this._timeoutId = 0;
    }
    if (this._panel) {
      Main.layoutManager.removeChrome(this._panel);
      this._panel.destroy();
      this._panel = null;
    }
    this._settings = null;
  }
}
```

4.4 Make big things own their cleanup

As the extension grows, `disable()` turns into a long list. The cleaner design, used by GNOME's own window-list extension, is: put the UI in its own class (a subclass of an St widget), connect signals with that widget as the owner, and clean up in the widget's `destroy` handler. Then `disable()` is just `this._windowList.destroy()`. Chapter 13 uses this pattern.

4.5 Other lifecycle rules

- **Nothing at import time.** Top-level code in your module runs when the file is loaded, not when enabled. Only define constants, functions and classes there; never create actors, connect signals or read settings at the top level.
- **`disable()` must be synchronous.** Since GNOME 51 an `async disable()` throws.
- **No constructor work.** Use `enable()`. The constructor may run once while `enable()` runs many times.
- **Session modes.** If you need to stay running on the lock screen (rare; a window list doesn't), declare `"session-modes": ["user", "unlock-dialog"]` in `metadata.json` and justify it in your review.

Chapter 5 — How to read the documentation

There is no single manual. There are three sources, each for a different question, and the API reference becomes readable once you know six translation rules.

5.1 Which source answers what

Question	Where to look
How do I get started, what is the lifecycle, how do preferences work?	gjs.guide/extensions — tutorials
What changed in my GNOME version?	gjs.guide port guides — read the ones for every version you support
What methods, properties and signals does `Meta.Window` / `St.Button` have?	gjs-docs.gnome.org — API reference for `gi://` libraries; pick the library version matching your GNOME
How does `Main.layoutManager` / `PopupMenu` / `DND` work?	The gnome-shell source, `js/ui/` — Shell's JS has no generated docs
How does a real extension do X?	gnome-shell-extensions — official extensions, including `window-list`
Can I publish this?	Review guidelines

In your editor, the `@girs` types are a fourth source: hover a name for its signature, or go-to-definition to read the generated `.d.ts`. That's often faster than the website.

5.2 Anatomy of a class page

Open any class on gjs-docs, for example `Meta.Window`. The page is split into sections:

- **Hierarchy** — the parent classes. A method you can't find may live on a parent. `St.Button` → `St.Bin` → `St.Widget` → `Clutter.Actor` → `GObject.Object`, so `St.Button` has `ease()`, `destroy()`, `add_child()` and everything else from `Clutter.Actor`.
- **Constructors** — how to create one. Many classes have none listed; you don't create them, you get them (windows come from `global.display`).
- **Properties** — values you read or write: `window.title`, `actor.width`. Each one also emits `notify::<name>` when it changes.
- **Signals** — events you can `connect()` to, with the callback's arguments.
- **Methods** — functions you call on an instance.
- **Static methods / functions** — called on the class itself, `Clutter.Settings.get_default()`. These are the exception; most methods are instance methods.

5.3 The six translation rules

1. The first C argument becomes `this`. The C function `meta_window_get_title(MetaWindow *window)` is `window.get_title()` in JS.

2. Names keep underscores for methods; properties work both ways. Methods stay `snake_case`: `get_focus_window()`. Properties are `kebab-case` in the docs (`focus-window`) but in JS you can write `display.focus_window` or `display.focusWindow`. Signal names stay as strings in kebab-case: `"notify::focus-window"`, `"window-created"`.

3. Getter methods and properties are usually the same thing. `window.get_title()` and `window.title` return the same value. Use whichever reads better.

4. Out-parameters become a returned array. C often "returns" several values by writing into pointers. GJS returns them as an array instead:

```
// C:  clutter_actor_get_preferred_width(actor, for_height, &min, &natural)
const [minWidth, naturalWidth] = actor.get_preferred_width(-1);

// C:  returns gboolean and writes to an out param
const [ok, value] = someObject.lookup_something(key);
```

5. Enums and flags are nested objects. `Clutter.ActorAlign.CENTER`, `Meta.KeyBindingFlags.NONE`. Combine flags with `|`: `Shell.ActionMode.NORMAL | Shell.ActionMode.OVERVIEW`.

6. Constructors take one object of properties. Instead of C's long argument list, GObject classes accept a dictionary of property values:

```
const label = new St.Label({
  text: "Hello",
  style_class: "my-label",
  y_align: Clutter.ActorAlign.CENTER,
});
```

Any writable property from the page (or a parent's page) can go in that object.

5.4 Reading Shell's JavaScript source

For everything under `resource:///org/gnome/shell/`, the source *is* the documentation. The path maps directly: `resource:///org/gnome/shell/ui/layout.js` is `js/ui/layout.js` in the gnome-shell repo. Always switch the repo to the tag for your version (for example `51.0`), since this code changes every release.

Useful habit: when you want to do something the Shell already does, find where Shell does it and copy the pattern. Want to know how the dash activates a window? Search `js/ui` for `activate(global.get_current_time())`.

5.5 Worked example: finding "window was closed"

1. Open `Meta.Window` on gjs-docs, scroll to **Signals**.
2. Find `unmanaged` (fired when Mutter stops managing the window). That's the closed event.
3. Callback signature is `(window) => void`. So: `metaWindow.connect("unmanaged", (w) => { … })`.
4. Check how GNOME uses it: search gnome-shell-extensions for `'unmanaged'`, and you'll find `window-list` removing its button there.

The same four steps work for any question: find the class, scan its signals and properties, check a real usage.

Chapter 6 — GObject essentials: classes, properties, signals

Every object from a `gi://` import is a GObject. GObject gives every object the same three features: **properties** (values), **signals** (events) and **class registration** (so C knows about your JS subclass). Learn these once and the whole API becomes predictable.

6.1 Signals: how anything "updates automatically"

Nothing in GNOME polls. Objects emit **signals** when something happens, and you register callbacks.

```
const id = global.display.connect("window-created", (display, window) => {
  console.log("new window:", window.get_title());
});

// later, to stop listening:
global.display.disconnect(id);
```

The callback's first argument is always the object that emitted the signal; the rest are listed on the signal's doc entry.

Some signals let you stop the event from going further. Event callbacks return `Clutter.EVENT_STOP` (handled, stop here) or `Clutter.EVENT_PROPAGATE` (let others see it too).

6.2 `notify::` — listening to property changes

Every property change emits a signal named `notify::<property-name>`. This is how you watch state:

```
// focus changed (covers "a new window opened and took focus")
global.display.connect("notify::focus-window", () => { … });

// a window's title changed (browser tab switch, file saved…)
window.connect("notify::title", () => { … });

// a window was minimized or restored
window.connect("notify::minimized", () => { … });
```

The property name in the signal is kebab-case, as in the docs, even though in JS you read it as `focus_window`.

6.3 `connectObject` and `disconnectObject`

Keeping every signal id and disconnecting each one is tedious and error-prone. GNOME Shell adds two helpers to every GObject:

```
// connect several signals at once, tagged with an owner (the last argument)
global.display.connectObject(
  "window-created", (_d, win) => this._addWindow(win),
  "notify::focus-window", () => this._updateFocus(),
  this,
);

// disconnect every handler that owner put on this object
global.display.disconnectObject(this);
```

Two extra benefits:

- If the **owner is a Clutter actor**, its handlers are disconnected automatically when the actor is destroyed. This is the cleanest way to avoid leaks: connect with your widget as the owner, and destroying the widget cleans everything up.
- If the **emitting object** is destroyed, handlers are dropped too, so you won't get callbacks from a dead object.

Use `connectObject` everywhere in new code. Plain `connect` is fine for one-off cases where you keep the id.

6.4 Properties

Read and write properties like normal fields. Some are read-only (the docs say so).

```
const title = window.title;              // read
label.text = "Hello";                    // write
actor.set({ opacity: 128, visible: true }); // write several at once
```

Bindings keep two properties in sync without writing signal handlers:

```
// whenever button.hover changes, set label.visible to the same value
button.bind_property("hover", label, "visible",
GObject.BindingFlags.SYNC_CREATE);
```

`SYNC_CREATE` copies the value immediately as well as on every change. `INVERT_BOOLEAN` flips it. Settings can bind the same way (Chapter 10).

6.5 Subclassing: `registerClass`

When you `extends` a GObject class (any St widget, any Clutter actor), you **must register** your subclass so the C side knows it exists. In TypeScript the cleanest form is a static block:

```
import GObject from "gi://GObject";
import St from "gi://St";

class WindowButton extends St.Button {
  static {
    GObject.registerClass(this);
  }

  private _window: Meta.Window;

  constructor(window: Meta.Window) {
    super({ style_class: "app-tab-button", can_focus: true });
    this._window = window;
  }
}
```

Rules for registered classes:

- Call `super({...})` with the property object first in your constructor.
- Older code uses `_init()` instead of `constructor`. Both work; prefer `constructor` in new code.
- Forgetting `registerClass` gives errors like "Tried to construct an object without a GType".
- Plain JS classes that don't extend a GObject (a helper, a manager) need no registration.

6.6 Custom properties and signals

Your own classes can declare GObject properties and signals, so other code can `connect` to them just like built-in ones:

```
class WindowList extends St.BoxLayout {
  static {
    GObject.registerClass({
      Properties: {
        "show-all-workspaces": GObject.ParamSpec.boolean(
          "show-all-workspaces", null, null,
          GObject.ParamFlags.READWRITE, false),
      },
      Signals: {
        "window-activated": { param_types: [GObject.TYPE_OBJECT] },
      },
    }, this);
  }
}

// elsewhere
list.emit("window-activated", someWindow);
list.connect("window-activated", (_list, win) => { … });
```

You won't need this on day one, but the window-list source uses it, so it helps to recognize it.

6.7 Object lifetime

When an actor is destroyed (`actor.destroy()`), its C side is freed but your JS reference still exists. Touching it afterwards logs "has already been disposed" or "impossible to access it" warnings. Defend against this by:

- setting fields to `null` after destroying,
- disconnecting signals on destroy (or using `connectObject` with an actor owner),
- listening to `"destroy"` on objects you keep references to: `actor.connect("destroy", () => (this._actor = null))`.

Windows have their own end-of-life signal: `unmanaged` on `Meta.Window` fires when a window closes. Drop your reference there.

Chapter 7 — Windows, workspaces and monitors

All window-manager state lives in Mutter (`Meta`), and you reach it through `global`. A window list needs four things from it: the list of windows, which one is focused, which workspace and monitor each is on, and signals for when any of that changes.

7.1 Getting the list of windows

There are two common ways, and they return different orders:

```
// 1) Every window actor; .metaWindow is the Meta.Window
const windows = global.get_window_actors().map((a) => a.metaWindow);

// 2) The Alt+Tab list for one workspace, most-recently-used first
const ws = global.workspace_manager.get_active_workspace();
const tabList = global.display.get_tab_list(Meta.TabList.NORMAL, ws);
```

For a stable, i3-style order (windows don't jump around when focus changes), sort by **stable sequence**, a number Mutter assigns when the window is created and never changes:

```
windows.sort((a, b) => a.get_stable_sequence() - b.get_stable_sequence());
```

The actor list includes things that don't belong in a task bar. Filter them:

```
const visible = windows.filter((w) => !w.skip_taskbar);
```

`skip_taskbar` is true for popups, tooltips, desktop icons and similar. `Meta.TabList.NORMAL` already excludes them.

7.2 `Meta.Window`: what you can read

Property / method	Gives you
`title` / `get_title()`	window title
`get_wm_class()`	the app's class, e.g. `"firefox"`
`get_id()`	numeric id, unique for this session
`get_stable_sequence()`	creation order, never changes
`minimized`	boolean
`is_maximized()`	boolean (GNOME 49 replaced `get_maximized()`)
`fullscreen`	boolean
`skip_taskbar`	should a task bar hide it?

Property / method	Gives you
`urgent`, `demands_attention`	wants the user's attention
`get_workspace()`	its `Meta.Workspace` (null if on all workspaces)
`located_on_workspace(ws)`	true if visible on that workspace, including "on all workspaces" windows
`is_on_all_workspaces()`	pinned to every workspace
`get_monitor()`	monitor index
`get_frame_rect()`	position and size as an `Mtk.Rectangle`
`get_transient_for()`	parent window for dialogs
`can_minimize()`, `can_maximize()`, `can_close()`	what's allowed

7.3 `Meta.Window`: what you can do

```
const time = global.get_current_time();

win.activate(time);            // focus it, switching workspace if needed
win.minimize();
win.unminimize();
win.maximize();                // GNOME 49+: no flags argument
win.unmaximize();
win.delete(time);              // politely ask it to close
win.change_workspace_by_index(2, false);
win.move_to_monitor(1);
win.move_resize_frame(true, x, y, width, height);
```

Methods that take `time` need it so Mutter can tell a real user action from a stale one; always pass `global.get_current_time()`.

7.4 Window signals you'll use

Object	Signal	Fires when
`global.display`	`window-created`	a new window appears (callback: `(display, window)`)
`global.display`	`notify::focus-window`	focus moves to another window, or to none
`global.display`	`window-entered-monitor`, `window-left-monitor`	a window moves between monitors
`Meta.Window`	`unmanaged`	the window closed
`Meta.Window`	`notify::title`	title changed
`Meta.Window`	`notify::minimized`	minimized or restored
`Meta.Window`	`workspace-changed`	moved to another workspace
`Meta.Window`	`notify::urgent`, `notify::demands-attention`	wants attention
`Meta.Window`	`notify::skip-taskbar`	task-bar visibility changed

A typical pattern: connect to `window-created` once on the display, then for each window connect to that window's own signals, and disconnect them on `unmanaged`.

One gotcha: `window-created` fires very early, sometimes before the title or app is known. Read those values in a `notify::title` handler rather than trusting the first value.

7.5 The focused window

```
const focused = global.display.focus_window; // Meta.Window or null
```

It is `null` when nothing has focus (for example, after clicking the desktop). Always handle `null`.

7.6 Workspaces

```
const wm = global.workspace_manager;

wm.n_workspaces;                          // count
wm.get_active_workspace();                // Meta.Workspace
wm.get_active_workspace_index();          // number
wm.get_workspace_by_index(1)?.activate(global.get_current_time());
wm.get_active_workspace().list_windows(); // windows on it
```

Signals: `global.window_manager` emits `switch-workspace` after a switch; `global.workspace_manager` emits `notify::n-workspaces` when workspaces are added or removed and `active-workspace-changed` on switches.

7.7 Monitors

Monitors come from `Main.layoutManager`, which gives you plain objects with geometry in global pixel coordinates:

```
const primary = Main.layoutManager.primaryMonitor;   // { index, x, y, width,
height, … }
const all = Main.layoutManager.monitors;             // array of the same
```

Listen for `monitors-changed` on `Main.layoutManager` to react to plugging in or resizing a screen. (Mutter also exposes newer `Meta.LogicalMonitor` objects since GNOME 49, but `Main.layoutManager` is still what Shell UI uses.)

7.8 Windows to apps: `Shell.WindowTracker`

To show an app icon or group windows by app, map windows to `Shell.App` objects:

```
const tracker = Shell.WindowTracker.get_default();
const app = tracker.get_window_app(win);       // Shell.App or null

app?.get_name();                               // "Firefox"
app?.create_icon_texture(24);                  // an actor you can add to
your UI
app?.get_windows();                            // all its windows
tracker.focus_app;                             // the focused Shell.App
```

`Shell.AppSystem.get_default().get_running()` lists running apps, and its `app-state-changed` signal tells you when one starts or stops.

Chapter 8 — Building UI: actors, St widgets, layout and CSS

Everything GNOME Shell draws is a tree of **actors**. You build UI by creating St widgets (which are actors), nesting them with `add_child()`, and attaching the root to the stage. Layout is done by **layout managers**, looks are done by **CSS**.

8.1 The actor tree

`global.stage` is the root. Under it, Shell keeps `Main.uiGroup`, which holds the panel, the overview, menus and anything you add with `Main.layoutManager.addChrome()`. Your widget becomes a branch of this tree:

```
global.stage
└── Main.uiGroup
    ├── panelBox (top bar)
    ├── … overview, windows, etc.
    └── your St.BoxLayout          ← addChrome() puts it here
        ├── St.Button  (window 1)
        │   └── St.BoxLayout
        │       ├── St.Icon
        │       └── St.Label
        └── St.Button  (window 2)
```

Core actor methods (all St widgets have them):

Method	Does
`add_child(a)` / `remove_child(a)`	build the tree
`insert_child_at_index(a, i)`, `set_child_at_index(a, i)`	control order
`get_children()`, `get_n_children()`	read the tree
`destroy()`	destroy this actor **and all its children**
`destroy_all_children()`	clear its contents
`show()` / `hide()`, `visible`	visibility
`get_parent()`, `contains(a)`	navigate

8.2 The St widgets you'll use

Widget	Use
`St.Widget`	generic container; set a `layout_manager` yourself
`St.BoxLayout`	a row (or column) of children; the workhorse
`St.Bin`	holds exactly one `child`; good for padding or centering
`St.Button`	clickable; one `child`, emits `clicked`
`St.Label`	text
`St.Icon`	a themed icon by name: `new St.Icon({ icon_name: "window-close-symbolic", icon_size: 16 })`
`St.Entry`	text input
`St.ScrollView`	scrollable area

8.3 Rows and columns

`St.BoxLayout` is horizontal by default. For vertical, set the orientation (the old `vertical: true` property was removed in GNOME 51):

```
const row = new St.BoxLayout();                                    //
horizontal
const col = new St.BoxLayout({ orientation: Clutter.Orientation.VERTICAL });
```

Spacing between children is set in CSS (`spacing: 4px;`).

8.4 Expand and align: how children share space

Each child says how it wants to be sized inside its parent:

Property	Meaning
`x_expand: true`	take any extra horizontal space the parent has
`y_expand: true`	same, vertically

Property	Meaning
`x_align` / `y_align`	where to sit inside the space it got: `Clutter.ActorAlign.START`, `CENTER`, `END`, or `FILL` (stretch)

A label vertically centered in a bar: `y_align: Clutter.ActorAlign.CENTER`. Buttons that share the width equally: give each `x_expand: true`. A spacer that pushes later items to the right: an empty `St.Widget({ x_expand: true })`.

8.5 Size and position

An actor's size normally comes from its content (its **preferred size**) and its position from its parent's layout manager. You only set them by hand for **top-level** actors you place on the stage, like your panel:

```
panel.set_size(width, height);
panel.set_position(x, y);
// or: panel.width = …; panel.x = …;
```

Coordinates are in global stage pixels: `(0, 0)` is the top-left of the whole desktop, and on multi-monitor setups each monitor sits at its own `x` / `y` offset. Bottom of the primary monitor:

```
const m = Main.layoutManager.primaryMonitor!;
panel.set_position(m.x, m.y + m.height - PANEL_HEIGHT);
panel.set_size(m.width, PANEL_HEIGHT);
```

Two traps:

- **Natural size isn't known until layout runs.** Reading `actor.height` right after creating it often gives 0. Either set a fixed size, or react to `notify::height` / `notify::allocation`.
- **Don't set sizes on children** inside a layout; use expand, align and CSS instead.

To measure without guessing: `const [min, natural] = actor.get_preferred_width(-1);`

8.6 Putting it on screen: `addChrome`

`Main.layoutManager.addChrome(actor, options)` adds your actor to Shell's UI and tells Shell how it relates to windows:

Option	Effect
`affectsStruts: true`	**reserves** the area so maximized windows stop at your panel instead of going under it
`trackFullscreen: true`	hide the actor while a fullscreen window is on that monitor
`affectsInputRegion: true` (default)	the actor receives clicks instead of the window below

Remove it with `Main.layoutManager.removeChrome(actor)` before destroying it. If you need to change options later, `untrackChrome(actor)` then `trackChrome(actor, newOptions)`.

Why the overview matters: struts and fullscreen tracking only make sense on the desktop. GNOME's window-list slides its panel out when `Main.overview` emits `showing` and back in on `hiding`; you'll likely want the same.

8.7 Styling with CSS

St widgets are styled with CSS. Give widgets a class and define it in `stylesheet.css` (loaded automatically, and unloaded on disable):

```
const panel = new St.BoxLayout({ style_class: "app-tab-panel" });
const button = new St.Button({ style_class: "app-tab-button" });
```

```
.app-tab-panel {
  background-color: rgba(30, 30, 30, 0.95);
  padding: 2px 8px;
  spacing: 4px;           /* St.BoxLayout-only: gap between children */
}

.app-tab-button {
  padding: 0 10px;
  border-radius: 6px;
  color: #ddd;
}

.app-tab-button:hover { background-color: rgba(255, 255, 255, 0.1); }
.app-tab-button:active { background-color: rgba(255, 255, 255, 0.2); }
.app-tab-button.focused { background-color: #3584e4; color: white; }
```

Things that differ from web CSS:

- It's a **subset**: no flexbox, grid or `display`. Layout is done in code (8.4).
- `spacing` is St-specific.
- Pseudo-classes `:hover`, `:active`, `:focus`, `:checked`, `:insensitive` are set automatically on St widgets that track them (buttons do). Add your own state with `add_style_class_name("focused")` / `remove_style_class_name("focused")`, or pseudo-classes with `add_style_pseudo_class("checked")`.
- Prefix every class with your extension's name. All extensions share one stylesheet namespace with the Shell theme.
- For quick experiments, inline `style: "background-color: red;"` works, but move it to the stylesheet before publishing.

Use Looking Glass's **Picker** (Chapter 3.3) on the real top bar to see which style classes Shell uses, and copy them if you want to match the system look.

8.8 Icons

```
// a themed icon by name (find names in /usr/share/icons/Adwaita)
new St.Icon({ icon_name: "view-app-grid-symbolic", icon_size: 16 });

// an app's own icon
Shell.WindowTracker.get_default().get_window_app(win)?.create_icon_texture(24
);

// an icon file shipped with your extension
new St.Icon({ gicon: Gio.icon_new_for_string(`${this.path}/icons/logo.svg`)
});
```

Chapter 9 — Interaction: clicks, scrolling, menus, animation

For a clickable thing, use `St.Button` and its `clicked` signal. For anything more (scrolling, long-press, keys), attach a Clutter **controller** to the actor. Animate with `actor.ease()`.

9.1 Buttons and clicks

```
const button = new St.Button({
  style_class: "app-tab-button",
  can_focus: true,                    // reachable with keyboard
  x_expand: true,
  button_mask: St.ButtonMask.PRIMARY | St.ButtonMask.SECONDARY,
  child: new St.Label({ text: win.title ?? "" }),
});

button.connect("clicked", (_btn, mouseButton: number) => {
  if (mouseButton === Clutter.BUTTON_SECONDARY) {
    menu.open();                     // right click → context menu
  } else {
    win.activate(global.get_current_time());
  }
});
```

`button_mask` decides which mouse buttons trigger `clicked`; by default only the primary one. GNOME 51 renamed `ONE` / `TWO` / `THREE` to `PRIMARY` / `MIDDLE` / `SECONDARY`; the old names still work, so use the old ones if you support GNOME 49 or 50.

Useful button properties: `hover` (read-only, with `notify::hover`), `checked` and `toggle_mode` for on/off buttons, `reactive: false` to make it ignore input.

9.2 Click-to-toggle-minimize (task bar behavior)

Task bars usually minimize a window if it's already focused, and activate it otherwise:

```
function toggle(win: Meta.Window) {
  if (global.display.focus_window === win && win.can_minimize())
    win.minimize();
  else
    win.activate(global.get_current_time());
}
```

9.3 Scroll, long press and keys: controllers

Since GNOME 51, input on an actor is handled by attaching **controllers**, objects that recognize one kind of input. Connecting to raw event signals like `scroll-event` still works but is deprecated.

```
// scroll wheel over the list switches windows
const scroll = new Clutter.ScrollController({
  flags: Clutter.ScrollControllerFlags.DISCRETE |
         Clutter.ScrollControllerFlags.SCROLL_VERTICAL,
});
scroll.connect("scroll", (_c, _sprite, _source, _dx, dy) => {
  this._focusNeighbor(dy > 0 ? 1 : -1);
});
this._list.add_action(scroll);
```

Other controllers: `Clutter.ClickGesture`, `Clutter.LongPressGesture`, `Clutter.KeyController`, `Clutter.MotionController`. Remove one with `actor.remove_action(controller)`. If you target GNOME 49 or 50 too, check the port guide for that version: the controller API is newer than the signals, and you may need the old `scroll-event` signal there.

The actor must be `reactive: true` to receive any input; `St.Button` is reactive by default, a plain `St.BoxLayout` is not.

9.4 Popup menus

Shell's own menus come from `ui/popupMenu.js`. A context menu that opens above a button:

```
import * as PopupMenu from "resource:///org/gnome/shell/ui/popupMenu.js";

// create (side BOTTOM = the menu's arrow points down at the source, so it
opens upward)
const menu = new PopupMenu.PopupMenu(button, 0.5, St.Side.BOTTOM);
menu.addAction("Close", () => win.delete(global.get_current_time()));
menu.addMenuItem(new PopupMenu.PopupSeparatorMenuItem());
menu.addAction("Minimize", () => win.minimize());

Main.uiGroup.add_child(menu.actor);   // menus live in uiGroup
menu.actor.hide();

// a manager handles grabs, Escape and closing other menus
const manager = new PopupMenu.PopupMenuManager(button);
manager.addMenu(menu);

menu.open();          // or menu.toggle()
```

Clean up with `menu.destroy()`. Since GNOME 51, `open()` and `close()` accept an options object (`menu.close({ animate: false })`) instead of an animation argument.

If you want a button **in the top bar** with a menu, use `PanelMenu.Button` from `ui/panelMenu.js` and `Main.panel.addToStatusArea(uuid, button)` instead; it wires all of this for you.

9.5 Animation with `ease()`

Any numeric actor property can be animated:

```
actor.ease({
  opacity: 0,
  translation_y: 32,
  duration: 250,                                  // ms
  mode: Clutter.AnimationMode.EASE_OUT_QUAD,
  onComplete: () => actor.hide(),
});
```

- Animate `translation_x/y`, `scale_x/y` and `opacity` rather than `x`, `y`, `width`: translations don't trigger re-layout and stay smooth.
- Starting a new `ease` on the same property cancels the old one. `actor.remove_all_transitions()` stops everything.
- GNOME 50 added `await actor.easeAsync({...})`, which resolves when done and rejects if interrupted.
- Shell scales durations by the user's animation settings and turns them off when animations are disabled, so you don't have to check. For motion-heavy effects, GNOME 51's `St.Settings.get().reducedMotion` tells you whether to tone it down.

9.6 Keyboard focus and accessibility

- Set `accessible_name` on buttons that only show an icon, so screen readers can announce them.
- Set `label_actor` to the label inside a button, so focus and screen readers use its text.
- `can_focus: true` plus a sensible order lets keyboard users Tab through your panel.

Chapter 10 — Settings (GSettings) and the preferences window

Settings are stored by **GSettings**: you declare keys in an XML schema, your extension reads them and listens for changes, and `prefs.ts` writes them. It's also the only channel between your extension and its preferences window.

10.1 Declare a schema

`schemas/org.gnome.shell.extensions.app-tab.gschema.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<schemalist>
  <schema id="org.gnome.shell.extensions.app-tab"
          path="/org/gnome/shell/extensions/app-tab/">

    <key name="panel-height" type="i">
      <default>32</default>
      <summary>Panel height in pixels</summary>
    </key>

    <key name="show-all-workspaces" type="b">
      <default>false</default>
```

```
      <summary>Show windows from every workspace</summary>
    </key>

    <key name="position" type="s">
      <default>"bottom"</default>
      <summary>Panel position: top or bottom</summary>
    </key>

  </schema>
</schemalist>
```

Rules: the file name is `<schema id>.gschema.xml`; the `id` matches `settings-schema` in `metadata.json`; the `path` starts and ends with `/`; key names are lowercase-with-dashes; string defaults keep their inner quotes (`"bottom"`).

Common types:

Type	Meaning	Read / write
`b`	boolean	`get_boolean` / `set_boolean`
`i`	integer	`get_int` / `set_int`
`d`	double	`get_double` / `set_double`
`s`	string	`get_string` / `set_string`
`as`	array of strings (used for keybindings)	`get_strv` / `set_strv`
enum	a named choice; declare `<enum>` in the schema and use `enum="…"` on the key	`get_enum` / `set_enum`

Compile before installing (the Makefile does this): `glib-compile-schemas schemas/`.

10.2 Read settings in the extension

```
enable(): void {
  this._settings = this.getSettings();          // uses settings-schema from
metadata.json
  const height = this._settings.get_int("panel-height");

  // react to changes made in prefs (or by dconf)
  this._settings.connectObject(
    "changed::panel-height", () => this._reposition(),
    "changed::show-all-workspaces", () => this._rebuild(),
    this,
  );
}

disable(): void {
  this._settings?.disconnectObject(this);
  this._settings = null;
}
```

Note the signal is `changed::<key>`, not `notify::`.

10.3 Bind a setting directly to a property

If a setting maps 1:1 to a GObject property, skip the handler:

```
// keeps button.ignore_workspace equal to the setting, now and on every
change
this._settings.bind("show-all-workspaces", button, "ignore-workspace",
  Gio.SettingsBindFlags.GET);
```

`GET` means settings → property only. `DEFAULT` is both ways (what prefs widgets use).

10.4 The preferences window

`prefs.ts` runs in a separate GTK 4 / libadwaita process, so it uses `Adw` and `Gtk`, never St or Clutter:

```
import Adw from "gi://Adw";
import Gio from "gi://Gio";
import Gtk from "gi://Gtk";
import { ExtensionPreferences, gettext as _ } from
  "resource:///org/gnome/Shell/Extensions/js/extensions/prefs.js";
```

```
export default class AppTabPrefs extends ExtensionPreferences {
  async fillPreferencesWindow(window: Adw.PreferencesWindow): Promise<void> {
    const settings = this.getSettings();

    const page = new Adw.PreferencesPage({ title: _("General") });
    const group = new Adw.PreferencesGroup({ title: _("Panel") });
    page.add(group);

    const allWs = new Adw.SwitchRow({ title: _("Show windows from all
workspaces") });
    group.add(allWs);
    settings.bind("show-all-workspaces", allWs, "active",
Gio.SettingsBindFlags.DEFAULT);

    const height = new Adw.SpinRow({
      title: _("Panel height"),
      adjustment: new Gtk.Adjustment({ lower: 20, upper: 64, step_increment:
1 }),
    });
    group.add(height);
    settings.bind("panel-height", height, "value",
Gio.SettingsBindFlags.DEFAULT);

    window.add(page);
  }
}
```

Note the import path: prefs use `resource:///org/gnome/Shell/Extensions/js/extensions/prefs.js` (capital S, different from the extension's path). Open the window with `gnome-extensions prefs <uuid>`.

Useful rows: `Adw.SwitchRow` (boolean), `Adw.SpinRow` (number), `Adw.ComboRow` (choice), `Adw.EntryRow` (text), `Adw.ActionRow` (anything custom).

10.5 Inspect and reset settings from the terminal

```
SCHEMADIR=~/.local/share/gnome-shell/extensions/app-
tab@yourname.github.io/schemas
gsettings --schemadir $SCHEMADIR list-recursively
org.gnome.shell.extensions.app-tab
gsettings --schemadir $SCHEMADIR set org.gnome.shell.extensions.app-tab
panel-height 40
gsettings --schemadir $SCHEMADIR reset-recursively
org.gnome.shell.extensions.app-tab
```

Setting a key here fires `changed::` in your running extension, which is a quick way to test reactions before you've built a prefs window.

Chapter 11 — Keyboard shortcuts

A global shortcut is a GSettings key of type `as` (a list of accelerator strings) plus one call to `Main.wm.addKeybinding()`. Because the shortcut lives in settings, users can change it without touching your code.

11.1 Add the keys to your schema

```
<key name="focus-next-window" type="as">
  <default><![CDATA[['<Super>j']]]></default>
  <summary>Focus the next window in the list</summary>
</key>

<key name="focus-previous-window" type="as">
  <default><![CDATA[['<Super>k']]]></default>
  <summary>Focus the previous window in the list</summary>
</key>
```

Accelerator syntax: modifiers in angle brackets followed by a key name, e.g. `<Super>j`, `<Super><Shift>Right`, `<Control><Alt>1`. Key names follow GDK names (`Right`, `Return`, `space`, `Page_Down`). The `CDATA` wrapper avoids escaping the `<` and `>`. An empty list (`[]`) means "no shortcut".

Before picking defaults, check for clashes: `gsettings list-recursively | grep -i super`. GNOME uses many `<Super>` combinations already, and a clash means one binding silently doesn't fire.

11.2 Register and unregister

```
import Meta from "gi://Meta";
import Shell from "gi://Shell";
import * as Main from "resource:///org/gnome/shell/ui/main.js";

enable(): void {
  this._settings = this.getSettings();

  Main.wm.addKeybinding(
    "focus-next-window",                // key name in your schema
    this._settings,                     // the Gio.Settings that holds it
    Meta.KeyBindingFlags.NONE,
    Shell.ActionMode.NORMAL | Shell.ActionMode.OVERVIEW,
    () => this._focusNeighbor(+1),      // handler
  );

  Main.wm.addKeybinding(
    "focus-previous-window",
    this._settings,
    Meta.KeyBindingFlags.NONE,
    Shell.ActionMode.NORMAL | Shell.ActionMode.OVERVIEW,
    () => this._focusNeighbor(-1),
  );
}

disable(): void {
  Main.wm.removeKeybinding("focus-next-window");
  Main.wm.removeKeybinding("focus-previous-window");
  this._settings = null;
}
```

Argument	Meaning
name	the schema key; must exist or `addKeybinding` throws
settings	your `Gio.Settings`
flags	`Meta.KeyBindingFlags.NONE` usually; `IGNORE_AUTOREPEAT` if holding the key shouldn't repeat

Argument	Meaning
action mode	where the shortcut works: `NORMAL` (desktop), `OVERVIEW`, `POPUP`, `LOCK_SCREEN` … combine with `|`
handler	called when pressed

If the user changes the key in settings, Mutter picks up the new accelerator automatically; you don't need to re-register.

11.3 Letting users change it in prefs

There is no ready-made shortcut-recorder row in libadwaita. Simple options:

- Use an `Adw.EntryRow` where the user types `<Super>j`, and on apply do `settings.set_strv("focus-next-window", [text])`.
- Build a small recorder with a `Gtk.EventControllerKey` that captures the next key press and converts it with `Gtk.accelerator_name(keyval, state)`. Several published extensions do this; read one for a pattern.

11.4 Overriding a GNOME shortcut

If you want `<Super>j` and GNOME already uses it, don't remove GNOME's binding silently. Either pick a free default, or document the clash so users can change one of them. Changing other schemas' keys from your extension is something reviewers will question.

Chapter 12 — Timers, the main loop and async code

GNOME Shell runs on a single thread with a **main loop**: it handles an event, runs your callback, redraws, and repeats. Anything slow in your callback freezes the entire desktop, so never block, and every timer you start must be removed in `disable()`.

12.1 No `setTimeout` habits: use GLib sources

GJS does provide `setTimeout`, but extensions should use GLib's timers, because they return an id you can remove reliably on disable:

```
// repeat every 500 ms until the callback returns GLib.SOURCE_REMOVE
this._timerId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 500, () => {
  this._poll();
  return GLib.SOURCE_CONTINUE;          // or GLib.SOURCE_REMOVE to stop
});

// run once (GNOME 50+ has these one-shot helpers)
this._onceId = GLib.timeout_add_once(GLib.PRIORITY_DEFAULT, 500, () => {
  this._onceId = 0;
  this._doSomething();
});

// run when the loop is idle (after current work)
this._idleId = GLib.idle_add(GLib.PRIORITY_DEFAULT_IDLE, () => {
  this._idleId = 0;
  this._sync();
  return GLib.SOURCE_REMOVE;
});
```

Cleanup, always:

```
disable(): void {
  if (this._timerId) GLib.source_remove(this._timerId);
  this._timerId = 0;
}
```

A timeout still running after disable is one of the most common review rejections. Also reset the stored id to 0 inside one-shot callbacks, so you don't later remove a source that already finished.

For a window list you rarely need timers at all: signals tell you when things change. Reach for a timer only for delays (like "open after hovering 500 ms") or to batch many updates into one.

12.2 "Run this after layout": laters

Sometimes you need to act after Shell has laid out and before it paints, for example to read an actor's real size. Mutter's **laters** do that:

```
const laters = global.compositor.get_laters();
this._laterId = laters.add(Meta.LaterType.BEFORE_REDRAW, () => {
  this._laterId = 0;
  this._updateSizes();
  return GLib.SOURCE_REMOVE;
});

// cleanup
if (this._laterId) laters.remove(this._laterId);
```

12.3 Async code

I/O (files, D-Bus, subprocesses) must be asynchronous so the desktop doesn't freeze. Many Gio methods come in `_async` / `_finish` pairs; GJS can turn them into Promises:

```
Gio._promisify(Gio.File.prototype, "load_contents_async",
"load_contents_finish");

async _loadConfig(file: Gio.File): Promise<string> {
  const [bytes] = await file.load_contents_async(null);
  return new TextDecoder().decode(bytes);
}
```

Rules for async in extensions:

- Pass a `Gio.Cancellable` and cancel it in `disable()`, so pending work doesn't touch destroyed objects when it finishes.
- After every `await`, re-check that you're still enabled (`if (!this._panel) return;`).
- Never make `enable()` or `disable()` async. `disable()` being async throws since GNOME 51.
- Catch errors; an unhandled rejection only shows up in the log.

12.4 Batching updates

Opening ten windows at once fires ten `window-created` signals. If each one rebuilds your whole list, you rebuild ten times. A small idle-based debounce fixes that:

```
private _queueRebuild(): void {
  if (this._rebuildId) return;              // already queued
  this._rebuildId = GLib.idle_add(GLib.PRIORITY_DEFAULT_IDLE, () => {
    this._rebuildId = 0;
    this._rebuild();
    return GLib.SOURCE_REMOVE;
  });
}
```

Chapter 13 — Project: an i3-style window list

This chapter builds a working bottom panel with one button per window, focus highlighting, click to activate or minimize, middle-click to close, scroll to switch, and `Super+J` / `Super+K` to move focus. It uses only what Chapters 1 to 12 covered, in about 200 lines.

The design has three pieces, each owning its own cleanup:

Piece	Type	Job
`WindowButton`	`St.Button` subclass	shows one window; keeps its own label and focus style up to date
`WindowList`	`St.BoxLayout` subclass	the panel; adds and removes buttons, positions itself, handles scrolling
`AppTab`	`Extension`	creates the list, registers shortcuts, tears it all down

13.1 Schema and metadata

Add to your schema (Chapter 10) and make sure `settings-schema` in `metadata.json` matches its id:

```
<key name="panel-height" type="i">
  <default>32</default>
  <summary>Panel height</summary>
</key>
<key name="show-all-workspaces" type="b">
  <default>false</default>
  <summary>Show windows from every workspace</summary>
</key>
<key name="focus-next-window" type="as">
  <default><![CDATA[['<Super>j']]]></default>
  <summary>Focus next window</summary>
</key>
<key name="focus-previous-window" type="as">
  <default><![CDATA[['<Super>k']]]></default>
  <summary>Focus previous window</summary>
</key>
```

13.2 Imports and `WindowButton`

One button per `Meta.Window`. It listens to its own window and to focus, using itself as the owner in `connectObject`, so destroying the button removes every handler (Chapter 6.3).

```
import Clutter from "gi://Clutter";
import Gio from "gi://Gio";
import GObject from "gi://GObject";
import Meta from "gi://Meta";
import Shell from "gi://Shell";
import St from "gi://St";
import * as Main from "resource:///org/gnome/shell/ui/main.js";
import { Extension } from
"resource:///org/gnome/shell/extensions/extension.js";

const ICON_SIZE = 18;

class WindowButton extends St.Button {
  static {
    GObject.registerClass(this);
  }

  readonly metaWindow: Meta.Window;
  private _label: St.Label;
```

```
  constructor(metaWindow: Meta.Window) {
    super({
      style_class: "app-tab-button",
      can_focus: true,
      // ONE/TWO = primary/middle; still valid on GNOME 51
      button_mask: St.ButtonMask.ONE | St.ButtonMask.TWO,
    });
    this.metaWindow = metaWindow;

    const box = new St.BoxLayout({ style_class: "app-tab-button-box" });
    this.set_child(box);

    const app = Shell.WindowTracker.get_default().get_window_app(metaWindow);
    box.add_child(app
      ? app.create_icon_texture(ICON_SIZE)
      : new St.Icon({ icon_name: "application-x-executable", icon_size:
ICON_SIZE }));

    this._label = new St.Label({ y_align: Clutter.ActorAlign.CENTER });
    box.add_child(this._label);
    this.label_actor = this._label;

    metaWindow.connectObject(
      "notify::title", () => this._sync(),
      "notify::minimized", () => this._sync(),
      this,
    );
    global.display.connectObject("notify::focus-window", () => this._sync(),
this);

    this.connect("clicked", (_btn: St.Button, mouseButton: number) => {
      if (mouseButton === Clutter.BUTTON_MIDDLE)
        this.metaWindow.delete(global.get_current_time());
      else
        this.toggleWindow();
    });

    this._sync();
  }

  get isFocused(): boolean {
    return global.display.focus_window === this.metaWindow;
  }
```

```
  /** Focused → minimize; otherwise → activate (Chapter 9.2). */
  toggleWindow(): void {
    if (this.isFocused && this.metaWindow.can_minimize())
      this.metaWindow.minimize();
    else
      this.metaWindow.activate(global.get_current_time());
  }

  // named focusWindow, not activate, to avoid clashing with names on parent
classes
  focusWindow(): void {
    this.metaWindow.activate(global.get_current_time());
  }

  private _sync(): void {
    const title = this.metaWindow.title ?? "";
    this._label.text = this.metaWindow.minimized ? `[${title}]` : title;

    if (this.isFocused) this.add_style_class_name("focused");
    else this.remove_style_class_name("focused");
  }
}
```

13.3 `WindowList`: the panel

The panel owns everything about the list. All its connections use `this` (an actor) as the owner, so `destroy()` cleans them up.

```
class WindowList extends St.BoxLayout {
  static {
    GObject.registerClass(this);
  }

  private _settings: Gio.Settings;

  constructor(settings: Gio.Settings) {
    super({ style_class: "app-tab-panel", reactive: true });
    this._settings = settings;

    // 1. put it on screen, reserving space for maximized windows (8.6)
    Main.layoutManager.addChrome(this, { affectsStruts: true,
```

```
trackFullscreen: true });
    this._reposition();

    // 2. react to the outside world
    Main.layoutManager.connectObject("monitors-changed", () =>
this._reposition(), this);
    settings.connectObject(
      "changed::panel-height", () => this._reposition(),
      "changed::show-all-workspaces", () => this._syncVisibility(),
      this,
    );
    global.display.connectObject(
      "window-created", (_d: Meta.Display, win: Meta.Window) =>
this._addWindow(win),
      this,
    );
    global.workspace_manager.connectObject(
      "active-workspace-changed", () => this._syncVisibility(),
      this,
    );

    // 3. scroll over the panel to move focus (9.3)
    const scroll = new Clutter.ScrollController({
      flags: Clutter.ScrollControllerFlags.DISCRETE |
             Clutter.ScrollControllerFlags.SCROLL_VERTICAL,
    });
    scroll.connect("scroll", (_c, _sprite, _source, _dx, dy: number) => {
      if (dy !== 0) this.focusNeighbor(dy > 0 ? 1 : -1);
    });
    this.add_action(scroll);

    // 4. add the windows that already exist, oldest first (7.1)
    global.get_window_actors()
      .map((a) => a.metaWindow)
      .sort((a, b) => a.get_stable_sequence() - b.get_stable_sequence())
      .forEach((win) => this._addWindow(win));
  }

  private _reposition(): void {
    const m = Main.layoutManager.primaryMonitor;
    if (!m) return;
    const height = this._settings.get_int("panel-height");
    this.set_size(m.width, height);
```

```
    this.set_position(m.x, m.y + m.height - height);
  }

  private _buttons(): WindowButton[] {
    return this.get_children().filter((c): c is WindowButton => c instanceof
WindowButton);
  }

  private _addWindow(win: Meta.Window): void {
    if (this._buttons().some((b) => b.metaWindow === win)) return;

    const button = new WindowButton(win);
    this.add_child(button);

    // owner = button: these handlers die with the button
    win.connectObject(
      "unmanaged", () => button.destroy(),            // window closed →
remove button
      "workspace-changed", () => this._syncVisibility(),
      "notify::skip-taskbar", () => this._syncVisibility(),
      button,
    );

    this._syncVisibility();
  }

  private _syncVisibility(): void {
    const showAll = this._settings.get_boolean("show-all-workspaces");
    const ws = global.workspace_manager.get_active_workspace();

    for (const b of this._buttons()) {
      const w = b.metaWindow;
      b.visible = !w.skip_taskbar && (showAll || w.located_on_workspace(ws));
    }
  }

  /** Move focus left (-1) or right (+1) through the visible buttons,
wrapping. */
  focusNeighbor(delta: number): void {
    const visible = this._buttons().filter((b) => b.visible);
    if (visible.length === 0) return;

    const current = visible.findIndex((b) => b.isFocused);
```

```
    const next = current === -1
      ? 0
      : (current + delta + visible.length) % visible.length;
    visible[next].focusWindow();
  }
}
```

13.4 The extension class

The extension itself is now short: create the list, add shortcuts, and reverse both in `disable()`.

```
const SHORTCUTS = ["focus-next-window", "focus-previous-window"] as const;

export default class AppTab extends Extension {
  private _settings: Gio.Settings | null = null;
  private _list: WindowList | null = null;

  enable(): void {
    this._settings = this.getSettings();
    this._list = new WindowList(this._settings);

    const mode = Shell.ActionMode.NORMAL | Shell.ActionMode.OVERVIEW;
    Main.wm.addKeybinding("focus-next-window", this._settings,
      Meta.KeyBindingFlags.NONE, mode, () => this._list?.focusNeighbor(1));
    Main.wm.addKeybinding("focus-previous-window", this._settings,
      Meta.KeyBindingFlags.NONE, mode, () => this._list?.focusNeighbor(-1));
  }

  disable(): void {
    for (const name of SHORTCUTS) Main.wm.removeKeybinding(name);

    if (this._list) {
      Main.layoutManager.removeChrome(this._list);
      this._list.destroy();          // destroys every button and all their
handlers
      this._list = null;
    }
    this._settings = null;
  }
}
```

13.5 `stylesheet.css`

```
.app-tab-panel {
  background-color: #1e1e1e;
  padding: 3px 6px;
  spacing: 4px;
}

.app-tab-button {
  padding: 0 10px;
  border-radius: 6px;
  color: #cccccc;
  max-width: 240px;
}

.app-tab-button:hover { background-color: rgba(255, 255, 255, 0.08); }
.app-tab-button.focused { background-color: #3584e4; color: #ffffff; }

.app-tab-button-box { spacing: 6px; }
```

13.6 Trace: what happens when you open a window

1. Mutter creates a `Meta.Window` and `global.display` emits `window-created`.
2. `WindowList._addWindow` builds a `WindowButton`, appends it (so order = creation order), and connects `unmanaged`, `workspace-changed` and `notify::skip-taskbar` with the button as owner.
3. The new window takes focus, so `global.display` emits `notify::focus-window`. **Every** button's `_sync` runs; the new one adds `focused`, the old one removes it.
4. The app sets its title a moment later: `notify::title` fires on that window only, and its button updates its label.
5. You close it: `unmanaged` fires, the button is destroyed, and because it was the owner, all handlers tied to it disconnect automatically.

13.7 Where to take it next

Each of these is a self-contained exercise, and the chapter that helps is in brackets:

- **Overview behavior.** Slide the panel out while the overview is open, using `Main.overview`'s `showing` and `hidden` signals and `ease()` on `translation_y` [8.6, 9.5]. GNOME's window-list shows a careful version, including keeping struts stable.

- **Right-click menu.** Add a `PopupMenu` per button with Minimize, Maximize, Close [9.4].
- **Per-monitor lists.** Create one `WindowList` per entry in `Main.layoutManager.monitors`, and filter buttons with `win.get_monitor()` [7.7].
- **Attention hint.** Style a button differently when `urgent` or `demands_attention` is true [7.4].
- **Jump to window N.** Add `Super+1…9` shortcuts that activate the Nth visible button [11].
- **Drag to reorder.** Study `DND.makeDraggable` in the window-list source; the hardest feature on this list.
- **A prefs window** for panel height and the workspace option [10.4].

Chapter 14 — Pitfalls, good habits, publishing and porting

Most extension bugs come from a short list of mistakes, and most review rejections from an even shorter one. Know them before you write much code.

14.1 The classic mistakes

Mistake	What you see	Fix
Calling an instance method on a class	`is not a function`	get the instance from `global` / `Main` (1.5)
Not disconnecting signals	duplicate reactions after lock/unlock, errors from dead objects	`connectObject` with an owner; `disconnectObject` in disable (6.3)
Timeouts left running	errors after disable, review rejection	store the id, `GLib.source_remove` in disable (12.1)
Keeping references after destroy	"already disposed" warnings	set fields to `null` (6.7)
Work at import time	breaks on lock screen, review rejection	move it into `enable()` (4.5)
Assuming `focus_window` is set	`Cannot read properties of null`	handle `null` (7.5)
Reading size right after creation	width/height 0	fixed size, or wait for `notify::allocation` (8.5)

Mistake	What you see	Fix
Rebuilding everything per signal	stutter when many windows open	update only what changed, or batch with idle (12.4)
Mixing GTK into the extension	crash on load	GTK/Adw only in `prefs.ts` (1.3)
Unprefixed CSS classes	your styles leak into Shell or other extensions	prefix with your extension name (8.7)
Blocking I/O	the whole desktop freezes	async Gio + cancellable (12.3)

14.2 Habits that keep you out of trouble

- Keep the extension class thin. Put real logic in widget classes that clean up on `destroy` (Chapter 13).
- Write `disable()` at the same time as `enable()`, not afterwards.
- Lock and unlock the screen in your devkit session after every change. It runs disable then enable, and catches most leaks.
- Prefer `Main.layoutManager` monitor geometry and `global` state over caching; things change underneath you.
- Read the source for any Shell JS module you use (`js/ui/…`), at your version's tag.
- Use `console.debug` liberally while developing, and remove noisy logs before publishing.

14.3 Publishing on extensions.gnome.org

1. Read the review guidelines end to end. They are short and every rule is enforced.
2. Build the zip with `make pack` (Chapter 2.7). It must contain **compiled JavaScript**, `metadata.json`, `schemas/` and `stylesheet.css`, and nothing else: no `node_modules`, no `.ts` sources, no build files.
3. Make sure the compiled JS is readable. Reviewers read your code; minified or bundled output gets rejected. Plain `tsc` output is fine.
4. Upload at extensions.gnome.org/upload, or since GNOME 49 with `gnome-extensions upload --accept-tos`.
5. A human reviews it; expect comments to fix. Typical asks: missing cleanup, work at import time, unnecessary `session-modes`, unused code, logging spam.

The main reviewer checklist, in short: everything created in `enable()` is destroyed in `disable()`; nothing runs at import; no GTK in the extension; no synchronous I/O or subprocess calls on the main thread; no unnecessary privileges; the settings schema id starts with org.gnome.shell.extensions.

14.4 Every GNOME release can break you

GNOME Shell has no stable extension API. Shell's JS modules change every six months, and you're expected to test and update `shell-version` for each release. A routine for each new GNOME version:

1. Read that version's port guide.
2. Search your code for every API it mentions.
3. Update `@girs/*` packages to the matching version and fix type errors; they often point straight at removed APIs.
4. Test in a devkit session, add the version to `shell-version`, publish.

Recent changes that affect a window list:

Version	Change
49	`Meta.Rectangle` removed → `Mtk.Rectangle`; `maximize()` / `unmaximize()` lost their flags argument; `get_maximized()` → `is_maximized()`; `Clutter.ClickAction` / `TapAction` removed → `ClickGesture` / `LongPressGesture`; nested mode gone → `--devkit`
50	X11 session removed; `easeAsync()` added; `GLib.timeout_add_once()` and friends added
51	`vertical` property on St widgets removed → `orientation`; `St.ButtonMask` renamed (`PRIMARY` / `MIDDLE` / `SECONDARY`, old names still work); event controllers recommended over event signals; `PopupMenu.open()/close()` take an options object; `async disable()` throws

Supporting several versions at once is possible: check `Config.PACKAGE_VERSION` (from `resource:///org/gnome/shell/misc/config.js`) and branch, as described in Targeting Older GNOME. Keep the range small; each extra version is more testing.

Appendix — Cheat sheet and links

"How do I…?" quick reference

I want to…	Code
get the focused window	`global.display.focus_window`
list all windows	`global.get_window_actors().map(a => a.metaWindow)`
list windows on a workspace (MRU order)	`global.display.get_tab_list(Meta.TabList.NORMAL, ws)`
get the active workspace	`global.workspace_manager.get_active_workspace()`
focus a window	`win.activate(global.get_current_time())`
close a window	`win.delete(global.get_current_time())`
know when a window opens	`global.display` → `window-created`
know when a window closes	`win` → `unmanaged`
know when focus changes	`global.display` → `notify::focus-window`
know when the workspace changes	`global.workspace_manager` → `active-workspace-changed`
get a window's app and icon	`Shell.WindowTracker.get_default().get_window_app(win)?.create_icon_texture(24)`
get monitor geometry	`Main.layoutManager.primaryMonitor` / `.monitors`
put an actor on screen	`Main.layoutManager.addChrome(actor, { affectsStruts: true })`
add to the top bar	`Main.panel.addToStatusArea(uuid, panelMenuButton)`
animate	`actor.ease({ opacity: 0, duration: 200, mode: Clutter.AnimationMode.EASE_OUT_QUAD })`
read a setting	`this.getSettings().get_int("key")`

I want to…	Code
react to a setting	`settings.connectObject("changed::key", cb, this)`
add a shortcut	`Main.wm.addKeybinding(name, settings, Meta.KeyBindingFlags.NONE, Shell.ActionMode.NORMAL, cb)`
run later, once	`GLib.timeout_add_once(GLib.PRIORITY_DEFAULT, ms, cb)` (GNOME 50+)
show a notification	`Main.notify("Title", "Body")`
log	`console.log(...)`, read with `journalctl -f -o cat /usr/bin/gnome-shell`

Commands

Task	Command
build	`make`
install	`make install`
enable / disable	`gnome-extensions enable <uuid>` / `gnome-extensions disable <uuid>`
state and errors	`gnome-extensions info <uuid>`
open prefs	`gnome-extensions prefs <uuid>`
test safely	`dbus-run-session gnome-shell --devkit --wayland`
live console	Alt+F2 → `lg`
compile schemas	`glib-compile-schemas schemas/`
publish	`make pack`, then `gnome-extensions upload --accept-tos`

Links

- gjs.guide — Extensions: official tutorials, lifecycle, preferences, debugging

- gjs.guide — TypeScript and LSP: the setup this guide assumes
- gjs.guide — Port guides: per-version API changes (49, 50, 51 summarized in 14.4)
- Review guidelines and best practices
- gjs-docs.gnome.org: API reference for Meta, Clutter, St, Shell, Gio, GLib
- gnome-shell source, js/ui: the real documentation for `Main`, `PopupMenu`, `DND`
- gnome-shell-extensions: official extensions; `window-list` is the reference for this project
- extensions.gnome.org: publishing

API details in this guide were checked against the gjs.guide pages above as of October 2026 (GNOME 51).